VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Multidisciplinary Project (CO3107)

Project Report - Semester 252

"RubyAlert: An Intelligent AIoT-Driven Laboratory Monitoring and Early Warning System"

Instructor(s): Vo Tuan Binh, Ph.D.

Students: Le Hoang Chi Vi – 2353336 *(Group CC01, Team 29)*

HO CHI MINH CITY, MAY 2026

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my multidisciplinary project supervisor, Vo Tuan Binh, Ph.D., for his invaluable guidance, continuous support, and academic mentorship throughout the course of this semester. His insights and feedback on smart systems and embedded programming significantly enhanced the quality of the engineering and implementation of this project.

I would also like to extend my thanks to the Faculty of Computer Science and Engineering at Ho Chi Minh City University of Technology (VNU-HCM) for providing the resources, laboratories, and academic environment required to conduct this project. Lastly, I am grateful to my family and fellow peers for their unwavering encouragement and constructive discussions during this research work.

Abstract

This study introduces RubyAlert, an intelligent AIoT-driven laboratory monitoring and early warning system designed to address the safety limitations of traditional static IoT architectures. Integrating perception, network, and application layers, the system collects telemetry from temperature, humidity, and gas sensors at 10-second intervals and transmits it via Mosquitto MQTT.

A key contribution of this project is the integration of a multi-model AI routing engine (Gemini, LLaMA, Qwen, and Pollinations AI) that performs natural language semantic scanning to control physical actuators (ventilation fan and buzzer) without native function-calling APIs. Additionally, we implement cloud-independent edge automation algorithms, including Hysteresis and Rate-of-Change, to ensure the laboratory remains protected during network disconnects. Real-world testing demonstrates that RubyAlert achieves robust threat response times of under 2 seconds, maintaining continuous operations with 100% edge reliability.

List of Figures

List of Tables

Contents

Chapter 1

Introduction

Open-Source Project Repository: The entire codebase, including the simulated multi-model edge server, firmware scripts, and responsive dashboard files, is hosted publicly to support academic reproducibility and collaboration: https://github.com/ukulevi/rubyalert-aiot-lab

1.1 Background of the Study

Laboratory safety represents one of the most critical aspects of academic and industrial research environments. Universities, in particular, host a wide spectrum of activities ranging from routine chemical synthesis to high-power electronic prototyping, all conducted in shared, multi-purpose lab spaces that often lack dedicated, continuous monitoring infrastructure.

In Vietnam, past incidents of fire anomalies, harmful chemical inhalations, and undetected gas leaks in research laboratories highlight the severe deficiency of proactive, autonomous safety architectures. According to fire prevention authorities, laboratory fires caused by electrical faults and chemical spills accounted for a significant portion of campus safety incidents reported between 2020 and 2025. These events underscore the urgent need for intelligent, real-time monitoring systems that can detect threats before they escalate into catastrophic incidents.

Conventional IoT implementations deployed in laboratory environments are largely *static*: they display telemetry on isolated physical screens or simple web views without closed-loop remote control capabilities, natural-language interaction capacity, or predictive reasoning. Such systems can notify operators *after* a threshold is breached, but they cannot *anticipate* a developing hazard or *reason* about the appropriate corrective action.

The rapid rise of the *Artificial Intelligence of Things (AIoT)* paradigm allows engineers to merge low-cost edge microcontrollers with state-of-the-art Large Language Models (LLMs) such as Google Gemini, Meta LLaMA, and Groq-hosted models. This paradigm shifts safety monitoring from a classic passive/reactive rule-check model to a **cogni-**

tive, self-governing ecosystem where the system can understand natural-language commands, autonomously evaluate risk profiles, and proactively engage safety actuators before traditional threshold-based alarms would trigger.

This project, named **RubyAlert**, proposes and implements a complete AIoT laboratory safety system that combines real-time sensory telemetry, edge intelligence, cloud-based natural language understanding, and a bidirectional communication layer to create a truly intelligent laboratory guardian.

1.2 Research Objectives

The primary objectives of this study are defined as:

1. **Real-time Telemetry Collection:** Design and deploy a physical sensory node using the ESP32 microcontroller to continuously track Temperature, Humidity, and Gas/Smoke parameters via DHT22 and MQ-2 sensor inputs with a 10-second sampling interval.
2. **High-Availability Web Dashboard:** Build a responsive, modern Web Dashboard using TailwindCSS and Chart.js that displays low-latency telemetry streams via MQTT-over-WebSocket, provides live historical graphing, and offers direct actuator control through an intuitive toggle interface.
3. **Bi-directional Alerting Infrastructure:** Configure a Telegram Chatbot delivering unified alert notifications to laboratory operators while accommodating remote control commands (`/status`, `/fan_on`, `/fan_off`, `/report`, `/dashboard`) and free-form natural language queries.
4. **Cognitive AI Orchestration:** Program a multi-model AI integration layer supporting Google Gemini, Groq (LLaMA 3.1), local Ollama instances, and a free Pollinations AI fallback to act as a reactive lab assistant. The AI executes hardware configurations via Function Calling and Semantic Text Scanning, and autonomously initiates emergency controls (e.g., fan actuation) when relative risks emerge.
5. **Edge Fault-Tolerance:** Implement local rule-based safety mechanisms including hysteresis automation, rate-of-change spike detection, sensor fault isolation, and manual override priority to guarantee system reliability even when cloud services are unavailable.

1.3 Scope and Limitations

The research parameters and constraints are structured as follows:

1.3.1 Scope

- **Physical Deployment:** The system is designed for deployment in a singular, standard university chemistry/IoT laboratory room. The sensory node monitors three environmental parameters: temperature, relative humidity, and combustible gas concentration.
- **Actuation:** Actuation is implemented through a 5V single-channel relay module driving a DC ventilation fan, supplemented with an active piezoelectric warning buzzer for audible alerts.
- **Communication:** All MQTT communication between the ESP32 node, edge server, and dashboard is conducted through a locally hosted Mosquitto MQTT Broker, providing isolation and reliability for academic demonstrations.
- **AI Models:** The system supports four AI backends (Gemini, Groq, Ollama, Pollinations) with automatic fallback mechanisms to ensure 24/7 chatbot availability regardless of individual API quotas.

1.3.2 Limitations

- Telemetry parameters are focused on relative hazard thresholds rather than absolute molecular spectrometry measurements. The MQ-2 sensor provides a general combustible gas concentration reading, not chemical-specific identification.
- The web dashboard does not implement user authentication or role-based access control, as the system is intended for trusted, single-laboratory deployment.
- Long-term telemetry persistence (database logging) is not implemented in the current version; data is retained only in the Chart.js live buffer (last 30 readings).
- The system relies on Wi-Fi connectivity; operation in environments without reliable wireless infrastructure would require hardware modifications.

1.4 Report Structure

This report is organized into six chapters:

- **Chapter 1 – Introduction:** Provides the motivation, objectives, and scope of the project.
- **Chapter 2 – Foundational Knowledge:** Reviews the theoretical foundations of IoT architecture, MQTT protocol, hardware specifications, and AIoT integration.

- **Chapter 3 – System Design:** Presents the hardware schematic, system architecture, and edge automation algorithms.
- **Chapter 4 – Methodology and Implementation:** Details the firmware development, server-side logic, AI multi-model integration, web dashboard construction, and Telegram chatbot implementation.
- **Chapter 5 – Experimental Results:** Documents system testing evidence including dashboard screenshots, Telegram conversation logs, and a comprehensive verification matrix.
- **Chapter 6 – Conclusion:** Summarizes key contributions and outlines the future development roadmap.

Chapter 2

Foundational Knowledge

This chapter reviews the theoretical foundations and technology standards upon which the RubyAlert system is constructed. Understanding these fundamentals is essential for appreciating the design decisions documented in subsequent chapters.

2.1 Internet of Things (IoT) Architecture

The Internet of Things (IoT) refers to the network of physical objects embedded with sensors, software, and connectivity that enables them to collect and exchange data. The standard IoT reference architecture consists of three primary layers [**?**]:

1. **Perception Layer (Sensing):** This is the physical hardware layer that interfaces with the real world. It comprises microcontrollers, sensors, and actuators that capture environmental data and execute physical actions. In RubyAlert, this layer includes the ESP32 microcontroller interfaced with the DHT22 temperature/humidity sensor, the MQ-2 gas sensor, a 5V relay module, and a piezoelectric buzzer.
2. **Network Layer (Transport):** This layer handles the bidirectional communication of telemetry data and control commands between the perception layer and the application layer. RubyAlert employs multiple protocols at this layer:
    - **MQTT** over TCP/IP for sensor-to-server telemetry and fan control commands.
    - **MQTT over WebSocket (WSS)** for real-time dashboard data streaming.
    - **HTTPS** for Telegram Bot API communication and cloud AI API calls.
3. **Application Layer (Processing & Presentation):** This layer processes incoming data, applies business logic, and presents results to end users. In RubyAlert, this encompasses:

- The Python Edge Server (`smart_lab_system.py`) with rule engine and AI orchestration.
- The HTML5/TailwindCSS Web Dashboard with Chart.js visualization.
- The Telegram Chatbot with natural language understanding.

2.2 Communication Protocol: MQTT

For sensor-to-server data exchange, **MQTT (Message Queuing Telemetry Transport)** was chosen over standard HTTP REST APIs. MQTT is an OASIS standard messaging protocol designed for constrained devices and low-bandwidth, high-latency networks [**?**]. The key advantages that justified this selection are:

Criterion	**MQTT**	**HTTP REST**
Minimum Header Size	2 bytes	∼700+ bytes
Connection Model	Persistent TCP	Per-request TCP
Message Pattern	Publish/Subscribe	Request/Response
Real-time Capability	Native (event-driven)	Polling required
Power Consumption	Very Low	Moderate–High
Bidirectional Control	Yes (via topics)	Requires WebSocket

Table 2.1: Comparison of MQTT vs. HTTP REST for IoT Telemetry

MQTT operates on a *publish/subscribe* model mediated by a central **Broker**. In this architecture:

- The ESP32 node **publishes** telemetry JSON payloads to the topic `rubyalert/lab_1/telemetry`.
- The Edge Server and Web Dashboard **subscribe** to the same topic to receive real-time updates.
- Fan control commands are published to the topic `rubyalert/lab_1/fan` with payloads `"1"` (ON) or `"0"` (OFF).

The MQTT Broker used in this project is **Eclipse Mosquitto**, configured locally on the edge server with a custom port (1884 for TCP, 9002 for WebSocket) to avoid conflicts with other services. The Mosquitto configuration file specifies two listeners:

Code:

```
# local_mosquitto.conf
listener 1884
allow_anonymous true
listener 9002
protocol websockets
```

2.3 Core Hardware Modules

The following hardware components form the perception layer of the RubyAlert system:

2.3.1 ESP32 Microcontroller Unit (MCU)

The ESP32, manufactured by Espressif Systems [**?**], is a dual-core 32-bit Xtensa LX6 CPU running at up to 240 MHz. It integrates Wi-Fi 802.11 b/g/n and Bluetooth 4.2/BLE on a single chip, making it ideal for IoT applications requiring wireless connectivity. Key specifications relevant to this project include:

- **GPIO:** 34 programmable General-Purpose Input/Output pins supporting digital I/O, ADC, DAC, PWM, I2C, SPI, and UART interfaces.
- **ADC:** Two 12-bit Successive Approximation Register (SAR) ADCs (ADC1 and ADC2) providing analog-to-digital conversion across 18 channels with configurable attenuation up to 3.6V input range.
- **Memory:** 520 KB SRAM, 448 KB ROM, with support for external SPI flash up to 16 MB.
- **Programming:** Supports Arduino framework (C/C++) and MicroPython for rapid prototyping.

In this project, the ESP32 is programmed using **MicroPython**, which provides a high-level Python runtime directly on the microcontroller, enabling rapid development and easy debugging via REPL (Read-Eval-Print Loop).

2.3.2 DHT22 Digital Temperature and Humidity Sensor

The DHT22 (also known as AM2302) utilizes a capacitive humidity sensor and a negative temperature coefficient (NTC) thermistor to measure surrounding air conditions. It communicates via a proprietary single-wire digital protocol:

- **Temperature Range:** −40 to 80°C with accuracy of ±0.5°C.
- **Humidity Range:** 0–100% RH with accuracy of ±2% RH.
- **Sampling Period:** Minimum 2 seconds between consecutive reads.
- **Output:** 40-bit serial data (16-bit humidity + 16-bit temperature + 8-bit checksum).

The sensor is connected to GPIO 4 on the ESP32. The MicroPython `dht` library handles the timing-critical single-wire protocol automatically.

2.3.3 MQ-2 Gas and Smoke Sensor

The MQ-2 sensor features a tin dioxide (SnO_2) semiconductor sensing element. In clean air, SnO_2 has relatively high resistance. When exposed to combustible gases (LPG, Propane, Carbon Monoxide, Methane, and general Smoke), the presence of reducing gases lowers the surface resistance of SnO_2, producing an analog output voltage directly proportional to gas concentration.

Key specifications:

- **Detection Range:** 200–10,000 ppm for combustible gases.
- **Output:** Analog voltage (0–3.3V when configured with ESP32 ADC attenuation at 11dB).
- **Warm-up Time:** Approximately 20 seconds for stable readings.
- **Operating Voltage:** 5V DC for the heating element, with a separate analog output pin.

The analog output (AO pin) is connected to GPIO 34 on the ESP32, which is one of the input-only ADC1 channels. The ESP32 ADC attenuation is configured to `ADC.ATTN_11DB` to support the full 0–3.6V input range, mapping to digital values of 0–4095 (12-bit resolution).

2.3.4 5V Relay Module and DC Fan

The relay module provides electrical isolation between the ESP32's 3.3V logic and the 5V DC fan motor circuit. When GPIO 27 outputs a HIGH signal, the relay closes the fan circuit, activating the ventilation fan. This design protects the microcontroller from back-EMF generated by the motor's inductive load.

2.3.5 Active Piezoelectric Buzzer

The active buzzer connected to GPIO 26 contains an internal oscillator circuit, requiring only a DC voltage to produce a continuous tone. It is used for localized audible alerts when critical thresholds are breached, ensuring that laboratory personnel are immediately aware of hazardous conditions even if they are not monitoring digital displays.

2.4 Artificial Intelligence in IoT (AIoT)

Rather than executing static if-then scripts, the RubyAlert system embeds AI reasoning capabilities directly into the edge server. This section describes the conceptual framework.

2.4.1 Google Gemini AI and Function Calling

Google Gemini [?] is a family of multimodal large language models developed by Google DeepMind. The RubyAlert system utilizes the `gemini-2.0-flash` model variant, which provides:

- **Natural Language Understanding (NLU):** Ability to interpret free-form Vietnamese and English instructions and map them to structured intents.
- **Function Calling:** A mechanism where the model outputs structured JSON describing which tool to invoke and with what parameters, rather than generating free-text responses. This allows the AI to directly control physical actuators.
- **Context Window:** Up to 1 million tokens, enabling rich system prompts with environmental context.

2.4.2 Multi-Model AI Architecture

To overcome API quota limitations inherent to free-tier usage and ensure 24/7 chatbot availability, RubyAlert implements a **multi-model AI router** supporting four distinct backends:

Provider	Model	API Type	Requires Key?
Gemini	gemini-2.0-flash	Google GenAI SDK	Yes
Groq	llama-3.1-8b-instant	OpenAI-compatible REST	Yes
Ollama	qwen2.5:3b (configurable)	Local REST API	No
Pollinations	openai (GPT-4o-Mini)	Free REST API	No

Table 2.2: Supported AI Model Backends in RubyAlert

The active provider is selected via the `LLM_PROVIDER` environment variable in the configuration file, defaulting to `"free"` (Pollinations AI) for zero-cost, unlimited-quota operation.

2.4.3 Three-Tier AI System Model

The AI capabilities are structured into three operational tiers:

1. **Tier 1 – Reactive Assistant:** Responds to user queries about current sensor readings, generates environmental audit reports, and answers descriptive questions about laboratory safety protocols.
2. **Tier 2 – Agentic Actor:** Parses conversational intent from natural language (e.g., "it feels hot in here" → turn on fan) and actuates relays through Function Calling or Semantic Text Scanning.
3. **Tier 3 – Autonomous Supervisor:** Proactively evaluates early hazard escalation when gas levels enter a "risk zone" (above 60% of the critical threshold but below the hard limit) and autonomously activates ventilation with a 5-minute cooldown between AI-driven interventions to conserve API tokens.

Chapter 3

System Design

This chapter presents the complete system design, from hardware connections to software architecture and the algorithmic logic that governs automated responses.

3.1 Hardware Schematic & Connection Specifications

The ESP32 pin configuration is systematically mapped in Table **??**. Each connection was selected based on pin capability constraints (e.g., ADC1 channels for analog input, standard GPIO for digital output).

Component	**Sensor Pin**	**ESP32 GPIO**	**Signal Type**	**Notes**
DHT22	OUT	GPIO 4	Digital I/O	Single-wire protocol; requires 10kΩ pull-up resistor
MQ-2	AO (Analog Out)	GPIO 34	Analog Input (ADC1_CH6)	11dB attenuation for 0–3.6V range
Relay Module	IN	GPIO 27	Digital Output	Active HIGH configuration
Active Buzzer	Positive (+)	GPIO 26	Digital Output	Internal oscillator; DC-driven

Table 3.1: ESP32 Hardware Pin Connections

Power Supply Configuration: The ESP32 is powered via USB (5V), which also provides the 5V rail for the relay module and active buzzer through the VIN pin. The DHT22 operates at 3.3V supplied by the ESP32's onboard voltage regulator. The MQ-2 sensor's heating element requires a dedicated 5V supply, connected to the ESP32's VIN rail.

3.2 System Architecture

The complete RubyAlert system follows a distributed architecture with four major subsystems communicating through the MQTT Broker as a central message bus. Figure ?? illustrates the data flow between all components.

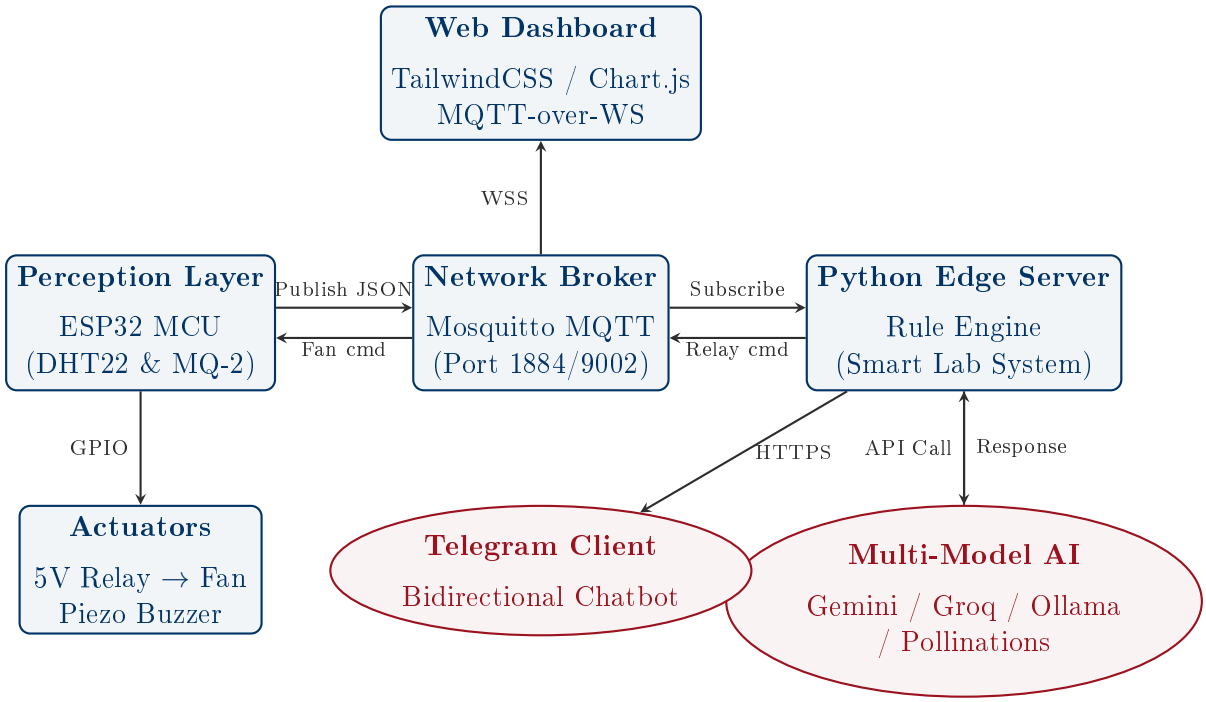

Figure 3.1: Vector Diagram of the RubyAlert Multi-tier AIoT Architecture

3.2.1 Data Flow Description

The end-to-end data flow operates as follows:

1. The **ESP32 Node** reads sensor data every 10 seconds, packages it into a JSON payload (`{"temperature": T, "humidity": H, "gas": G}`), and publishes it to the MQTT topic `rubyalert/lab_1/telemetry`.
2. The **Mosquitto Broker** relays the message to all subscribers: the Python Edge Server (via TCP on port 1884) and the Web Dashboard (via WebSocket on port 9002).
3. The **Edge Server** receives the telemetry, runs it through the local rule engine (hysteresis, rate-of-change, sensor fault checks), and publishes fan commands to `rubyalert/lab_1/fan` when thresholds are breached.
4. Simultaneously, the **Telegram polling thread** on the Edge Server checks for incoming user messages every 2 seconds. User commands are processed locally (for simple keywords) or forwarded to the AI backend (for complex queries).

5. The **Web Dashboard** renders incoming telemetry in real-time: updating metric cards, appending data points to the Chart.js graph, and logging system events.

3.3 Software Architecture

The server-side software is organized into modular Python files:

File	Purpose
`config.py`	Centralized configuration: API keys, MQTT broker address/port, LLM provider selection, MQTT topics. Loads from `.env` file.
`smart_lab_system.py`	Main edge server: MQTT client, telemetry processing, AIoT rule engine, Telegram bot, multi-model AI router, fan control functions.
`esp32_simulator.py`	Standalone ESP32 simulator that generates random telemetry data for testing without physical hardware.
`lab_analytics_backend.py`	Lightweight analytics backend that subscribes to all MQTT topics for debugging and threshold-based alerting.
`local_mosquitto.conf`	Mosquitto Broker configuration specifying TCP (1884) and WebSocket (9002) listeners.

Table 3.2: Server-side File Structure

3.4 Centralized Edge Automation Algorithms

To safeguard hardware integrity and ensure rapid reaction times independent of cloud availability, localized edge logic overrides cloud-dependent AI processing:

3.4.1 Hysteresis Automation

To prevent relay chattering—the rapid on/off switching of the fan near critical thresholds that can damage relay contacts and reduce fan motor lifespan—a hysteresis band is enforced. The system uses two separate threshold values:

$$T_{\text{activation}} = 35°\text{C} \quad \text{(Fan turns ON)} \tag{3.1}$$

$$T_{\text{deactivation}} = 32°\text{C} \quad \text{(Fan turns OFF)} \tag{3.2}$$

This creates a 3°C dead-band: once the fan is activated at 35°C, it remains active until

temperature drops below 32°C. Similarly, gas thresholds use a 60% deactivation ratio:

$$G_{\text{activation}} = 2000 \text{ ppm} \tag{3.3}$$
$$G_{\text{deactivation}} = 1200 \text{ ppm} \quad (= 2000 \times 0.6) \tag{3.4}$$

3.4.2 Rate of Change (RoC) Detection

Sudden thermal spikes indicative of chemical fires or electrical faults are detected by comparing consecutive temperature readings:

$$\text{RoC} = T_{\text{current}} - T_{t-10\text{s}} \tag{3.5}$$

If RoC ≥ 3°C within a single 10-second sampling interval, the system logs a spike event and appends a warning annotation to the Telegram alert message. This allows operators to distinguish between gradual temperature increases (e.g., HVAC changes) and potentially dangerous rapid rises.

3.4.3 Sensor Fault-Tolerance & Hardware-Safe Fallbacks

When the DHT22 sensor is physically disconnected or malfunctions, the MicroPython driver returns default values of 0.0°C and 0.0% humidity. The edge server detects this specific pattern:

Code:

```
# Sensor Fault Detection Logic
if t == 0.0 and h == 0.0:
    log("SENSOR_ERROR", "DHT22 fault detected")
    # Send hardware warning via Telegram (1-min cooldown)
    send_telegram("DHT22 sensor error. Check wiring.")
    # Preserve last valid temperature/humidity readings
    # Continue gas-only protection logic
    if g > GAS_LIMIT:
        client.publish(config.FEED_FAN, "1")
    return
```

This ensures that a single sensor failure does not disable the entire safety system. The gas monitoring subsystem continues operating independently with full threshold logic.

3.4.4 Manual Override Priority

To respect human authority over automated systems, manual toggle commands issued via the Web Dashboard or Telegram raise a `manual_override` flag. While this flag is set:

- The automatic hysteresis rule engine is **paused** for the fan actuator.
- Gas-related safety overrides still function (safety-critical exceptions).
- The flag is automatically cleared when a critical threshold override occurs, ensuring that safety always takes precedence over manual control.

3.4.5 Algorithmic Flowchart

The complete logic execution path of the localized edge computing platform is mapped in Figure **??**.

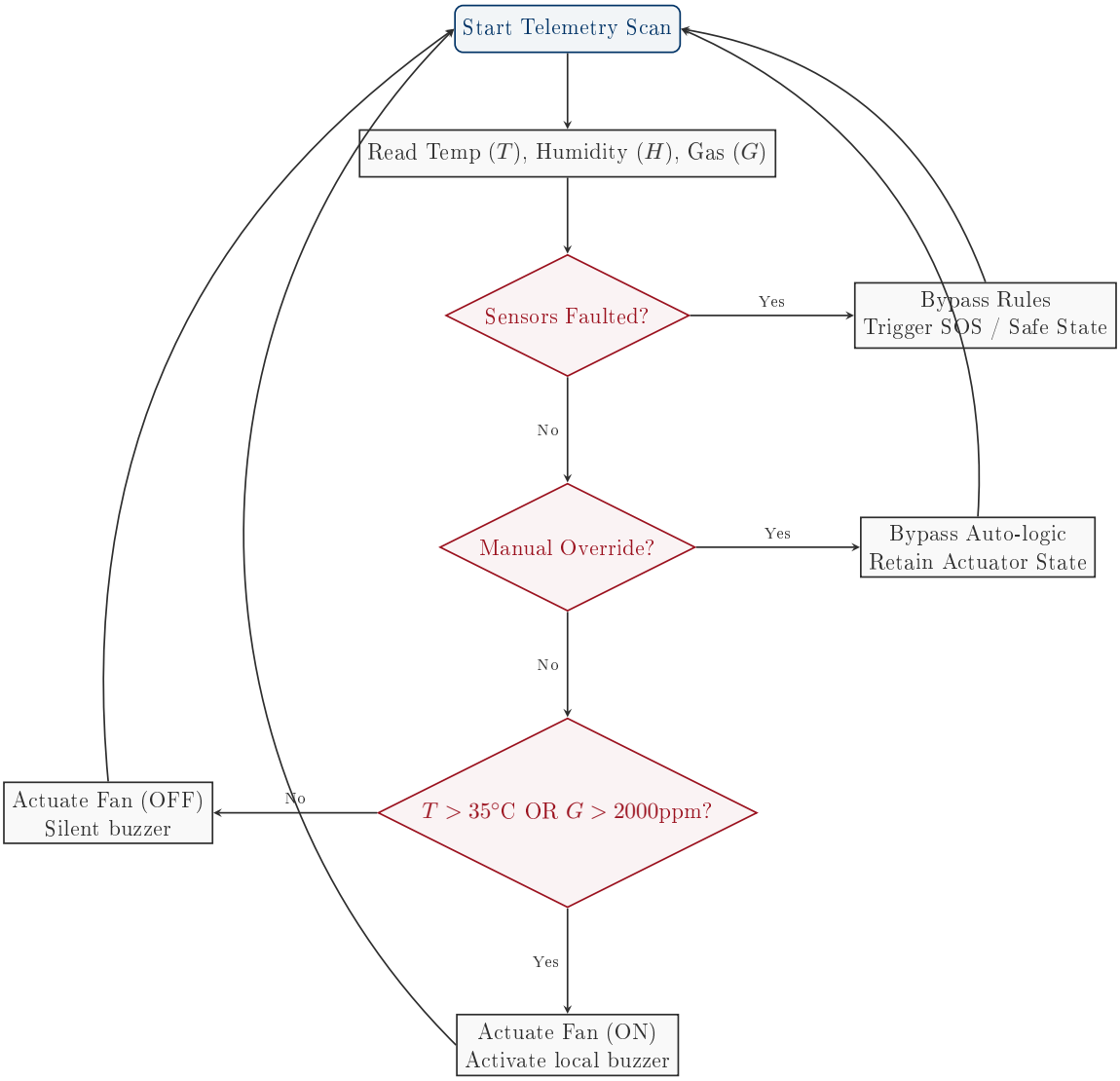

Figure 3.2: Central Local Core Edge Algorithm Execution Flowchart

Chapter 4

Methodology and Implementation

This chapter provides a detailed account of the implementation of each subsystem, including code excerpts, configuration details, and design rationale.

4.1 ESP32 Firmware Development (MicroPython)

The ESP32 firmware is written in **MicroPython** and stored in `firmware/main.py`. The firmware binary (`esp32_micropython.bin`) is flashed to the ESP32 using `esptool.py`. The firmware implements the following core functions:

4.1.1 Wi-Fi Connection Management

The `connect_wifi()` function activates the ESP32's Station Interface (STA_IF), attempts connection to the configured SSID, and waits up to 15 seconds with a polling loop before timing out. If the connection fails, the ESP32 performs a hard reset via `machine.reset()` after a 10-second delay, implementing an automatic retry mechanism:

Code:

```
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
        wlan.connect(WIFI_SSID, WIFI_PASS)
        count = 0
        while not wlan.isconnected() and count < 30:
            time.sleep(0.5)
            count += 1
    if wlan.isconnected():
        print("WiFi Connected! IP:", wlan.ifconfig()[0])
        return True
    else:
        return False
```

4.1.2 MQTT Client and Callback System

The firmware uses the `umqtt.simple` library to establish a persistent connection with the Mosquitto Broker. Upon connection, the ESP32 subscribes to the fan control topic (`rubyalert/lab_1/fan`) and registers a callback function `on_receive_message()` that processes incoming fan commands:

Code:

```
def on_receive_message(topic, msg):
    global fan_state
    if msg == b'1':
        relay_fan.on()     # Activate relay (Active HIGH)
        fan_state = 1
    else:
        relay_fan.off()    # Deactivate relay
        fan_state = 0
```

4.1.3 Main Telemetry Loop

The main loop (`run_real_hardware()`) runs continuously and performs the following operations every 10 seconds:

1. Checks Wi-Fi connectivity and reconnects if necessary.

2. Calls `client.check_msg()` non-blocking to process any pending fan commands.
3. Reads the DHT22 sensor via `sensor_dht.measure()`.
4. Reads the MQ-2 analog output via `sensor_mq2.read()` (0–4095).
5. Packages readings into a JSON payload and publishes to the telemetry topic.

Each sensor read is wrapped in individual try/except blocks to ensure that a failure in one sensor does not prevent reading from the other, implementing graceful per-sensor error handling.

4.2 Edge Server Implementation

The central edge server (`server/smart_lab_system.py`) is a Python 3 application running approximately 577 lines of code. It orchestrates all backend logic through a multi-threaded architecture.

4.2.1 Thread Architecture

The edge server operates three concurrent threads:

Thread	Type	Responsibility
Main Thread	Blocking	MQTT `loop_forever()` – processes all incoming MQTT messages (telemetry and fan commands)
Telegram Thread	Daemon	Long-polling the Telegram Bot API every 2 seconds for incoming user messages
Simulator Thread	Daemon (optional)	Generates random telemetry data when `SIMULATION_MODE = True` for testing without hardware

Table 4.1: Edge Server Thread Architecture

4.2.2 Telemetry Processing Pipeline

When a telemetry message arrives on the `rubyalert/lab_1/telemetry` topic, the `on_message()` callback parses the JSON payload and passes it to `process_aiot_logic(t, h, g)`, which implements the complete decision tree described in Section ??. The function:

1. Checks for DHT22 sensor faults (both temperature and humidity equal to 0.0).

2. Computes the Rate of Change against the previous temperature reading.
3. Evaluates hysteresis thresholds for both temperature and gas.
4. Generates and sends formatted Telegram alerts with appropriate severity levels.
5. Triggers the Tier 3 AI autonomous protection when gas enters the risk zone (60%–100% of threshold).

4.2.3 Telegram Alert System

The alert system implements intelligent notification management through:

- **60-second Cooldown:** Automatic alert messages are rate-limited to prevent notification flooding during sustained threshold breaches. A global timestamp tracks the last alert time.
- **Cooldown Bypass:** User-initiated responses (replies to commands) bypass the cooldown to ensure responsive interaction.
- **Markdown Formatting:** All Telegram messages use Markdown syntax for visual emphasis (**bold** for titles, `code` for numeric values).
- **Multi-tier Alert Severity:** Messages are categorized as dual-hazard (temperature + gas), single-hazard, or AI-proactive, each with distinct formatting and icons.

4.3 Multi-Model AI Integration

The AI integration layer is the most architecturally complex component of the system. It provides a unified interface (`ask_gemini_ai()`) that transparently routes queries to the configured AI backend.

4.3.1 Unified AI Router

The router function `ask_gemini_ai(user_question)` constructs a compressed system instruction containing real-time sensor data and behavioral rules, then dispatches the query based on the `LLM_PROVIDER` configuration:

Code:

```
def ask_gemini_ai(user_question):
    sys_instruction = f"""RubyAlert System.
    T:{last_temp}C, H:{last_humi}%, G:{last_gas}ppm,
    Fan:{"ON" if fan_on else "OFF"}.
    Rules: 1. Answer only about Lab safety/devices.
    2. Reject off-topic queries. 3. Be concise.
    4. Use "turn_on_fan" or "turn_off_fan" keywords."""

    provider = config.LLM_PROVIDER
    if provider == "groq":
        reply = ask_groq_ai(user_question, sys_instruction)
    elif provider == "ollama":
        reply = ask_ollama_ai(user_question, sys_instruction)
    elif provider == "free":
        reply = ask_pollinations_ai(user_question,
           sys_instruction)
    else:
        reply = ask_gemini_sdk_ai(user_question,
           sys_instruction)

    # Semantic Text Scanner for all models
    if "turn_on_fan" in reply.lower():
        turn_on_fan()
    elif "turn_off_fan" in reply.lower():
        turn_off_fan()
    return reply
```

4.3.2 Semantic Text Scanner

A critical innovation in this system is the **Semantic Text Scanner**. Not all AI models support native Function Calling (only Gemini does in this configuration). To enable fan control across all four AI backends, the system scans the AI's text response for predefined action keywords:

- Keywords for fan activation: "bật quạt", `"turn on the fan"`, `"turn_on_fan"`
- Keywords for fan deactivation: "tắt quạt", `"turn off the fan"`, `"turn_off_fan"`

The system instruction explicitly tells the AI to include these keywords in its response when it determines that fan action is appropriate. This approach provides cross-model compatibility without requiring each model to support structured output.

4.3.3 Gemini Function Calling Implementation

When using the Gemini provider, the system leverages native Function Calling. The AI model is provided with a tool list containing the `turn_on_fan()` and `turn_off_fan()` Python functions. When the model determines that an action is needed, it returns a structured `function_call` object instead of plain text:

Code:

```
# Gemini SDK with Function Calling
response = client_ai.models.generate_content(
    model='gemini-2.0-flash',
    contents=user_question,
    config=types.GenerateContentConfig(
        system_instruction=sys_instruction,
        tools=tools_list,  # [turn_on_fan, turn_off_fan]
        max_output_tokens=150
    )
)
# Check if AI returned a function call
if response.candidates[0].content.parts[0].function_call:
    fn = response.candidates[0].content.parts[0].
       function_call
    if fn.name == "turn_on_fan":
        return turn_on_fan()
```

4.3.4 Pollinations AI (Free Provider)

The Pollinations AI integration provides unlimited, keyless operation through a simple REST API:

Code:

```
def ask_pollinations_ai(user_question, sys_instruction):
    url = "https://text.pollinations.ai/"
    payload = {
        "messages": [
            {"role": "system", "content": sys_instruction},
            {"role": "user", "content": user_question}
        ],
        "model": "openai"
    }
    r = requests.post(url, json=payload, timeout=20)
    return r.text.strip()
```

4.4 Telegram Chatbot Implementation

The Telegram chatbot provides the primary user interface for remote monitoring and control. It implements both structured commands and natural language processing.

4.4.1 Command Registration

At startup, the bot registers a menu of predefined commands with Telegram's `setMyCommands` API, making them appear as autocomplete suggestions in the chat interface:

Command	Function
`/status`	Returns current temperature, humidity, gas, and fan state
`/fan_on`	Directly activates the exhaust fan via MQTT
`/fan_off`	Directly deactivates the exhaust fan via MQTT
`/report`	Triggers AI to generate a comprehensive safety assessment report
`/dashboard`	Returns the Web Dashboard access URL with server status

Table 4.2: Telegram Bot Command Menu

4.4.2 Hybrid Local/AI Message Processing

To optimize API token consumption, the chatbot implements a two-tier processing strategy:

1. **Local Keyword Matching:** Short, direct commands (under 15 characters) containing exact keywords like "bật quạt" (turn on fan) or "tắt quạt" (turn off fan) are processed entirely locally without invoking the AI model. This provides instant response times and zero API cost.

2. **AI Routing:** Complex, ambiguous, or conversational queries are forwarded to the configured AI backend for natural language understanding. The system also checks for negation words ("không", "đừng", "don't") and compound commands before routing, to avoid misinterpretation.

Code:

```
# Hybrid processing logic
negations = ["khong", "dung", "don't", "no"]
has_negation = any(neg in text for neg in negations)
is_short = len(text.strip()) <= 15

if not has_negation and is_short:
    for keyword, func in quick_replies.items():
        if keyword in text:
            reply = func()  # Local processing
            break
else:
    reply = ask_gemini_ai(text)  # AI processing
```

4.5 Web Dashboard Implementation

The Web Dashboard (`dashboard/dashboard.html`) is a single-page application providing real-time visualization and control. It is implemented as a self-contained HTML file with embedded CSS (via TailwindCSS CDN) and JavaScript.

4.5.1 Technology Stack

- **Layout & Styling:** TailwindCSS with a custom Material Design 3-inspired color palette featuring 40+ semantic color tokens.
- **Typography:** Google Fonts (Inter) with multiple weight variants for hierarchy.
- **Charting:** Chart.js for real-time line graphs with dual Y-axes (temperature/humidity on left, gas on right).
- **MQTT Client:** MQTT.js library connecting via WebSocket (`ws://host:9002/mqtt`).
- **Icons:** Google Material Symbols Outlined with variable font settings.

4.5.2 Dashboard Features

The dashboard interface includes:

1. **Connection Status Indicator:** A pulsing dot (red = offline, green = online) with the last sync timestamp.
2. **Metric Cards (4 cards):**
   - Temperature card with min/max display and color transition (white → red) on threshold breach.
   - Humidity card with average and dew point calculations.
   - Gas Level card with critical threshold footer and alert state styling.
   - Exhaust Fan card with animated spinning icon and hardware toggle switch.
3. **Live Telemetry Chart:** A 30-point rolling line chart with three datasets (Temperature, Humidity, Gas) rendered in real-time as MQTT messages arrive.
4. **System Event Log:** A self-pruning log panel (maximum 5 entries) categorized by event type (System, Alert, Action) with color-coded badges and timestamps.
5. **Responsive Navigation:** Desktop sidebar navigation and mobile bottom tab bar for cross-device compatibility.

4.5.3 Real-time MQTT Integration

The dashboard connects to the Mosquitto Broker via WebSocket using auto-detected host addressing:

Code:

```
// Auto-detect broker host for local/remote access
const brokerHost = window.location.hostname || '127.0.0.1';
const brokerUrl = `ws://${brokerHost}:9002/mqtt`;

const client = mqtt.connect(brokerUrl, {
    clientId: "RubyAlert_Web_" + Math.random().toString(16),
    clean: true,
    connectTimeout: 5000,
    reconnectPeriod: 3000,  // Auto-reconnect every 3s
});

client.on('message', function (topic, message) {
    if (topic === TOPIC_TELEMETRY) {
        const data = JSON.parse(message.toString());
        updateDashboard(data.temperature, data.humidity, data
            .gas);
    } else if (topic === TOPIC_FAN) {
        updateFanUI(message.toString() === "1");
    }
});
```

4.5.4 Fan Toggle Control

The dashboard provides a custom-styled toggle switch that publishes MQTT messages directly to the fan control topic, enabling real-time bidirectional control:

Code:

```
function toggleFan() {
    const isChecked = document.getElementById('fan-switch').
        checked;
    client.publish(TOPIC_FAN, isChecked ? "1" : "0");
    addLog('Action', `Fan turned ${isChecked ? 'ON' : 'OFF'
        }.`);
}
```

4.6 Configuration and Deployment

4.6.1 Environment Variables

All sensitive credentials and configurable parameters are stored in a `.env` file (excluded from version control via `.gitignore`). The `config.py` module loads these using `python-dotenv`:

Variable	Purpose
`TELEGRAM_TOKEN`	Telegram Bot API token from BotFather
`TELEGRAM_CHAT_ID`	Authorized chat ID for message filtering
`GEMINI_API_KEY`	Google AI Studio API key for Gemini
`LLM_PROVIDER`	AI backend selector (free/gemini/groq/ollama)
`GROQ_API_KEY`	Groq Cloud API key (optional)
`OLLAMA_HOST`	Local Ollama server URL (default: localhost:11434)
`OLLAMA_MODEL`	Ollama model name (default: qwen2.5:3b)

Table 4.3: Environment Variable Configuration

4.6.2 Deployment Steps

The system deployment follows this sequence:

1. Flash the ESP32 with MicroPython firmware (`firmware/esp32_micropython.bin`).
2. Upload `firmware/main.py` to the ESP32's filesystem.
3. Install Python dependencies: `pip install -r requirements.txt`.
4. Start the Mosquitto Broker: `mosquitto -c server/local_mosquitto.conf`.
5. Configure `.env` with API keys and credentials.
6. Start the edge server: `python server/smart_lab_system.py`.
7. Open the dashboard: `dashboard/dashboard.html` in a web browser.

Chapter 5

Experimental Results

This chapter presents empirical evidence of the RubyAlert system in active operation, including visual verification through screenshots, Telegram conversation logs, and a comprehensive system verification matrix.

5.1 Visual Verification & Dashboard Evidence

To provide empirical evidence of the system in active runtime, the physical deployment and rendering of the Web Dashboard are demonstrated across multiple operational states.

5.1.1 Initial Connection State

Figure **??** shows the Web Dashboard immediately after establishing a successful MQTT WebSocket connection. The status indicator transitions from red (Offline) to green (Online), confirming bidirectional communication with the Mosquitto Broker.

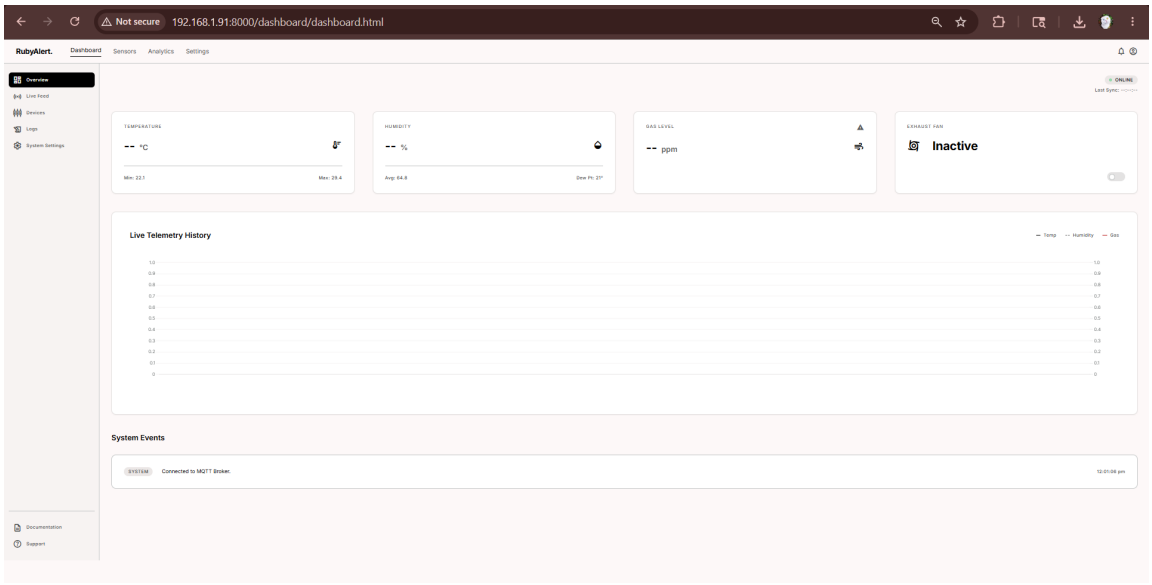

Figure 5.1: Web Dashboard State: MQTT WebSocket Connection Initialized (Status: Online)

5.1.2 Normal Telemetry Stream

Figure **??** demonstrates the dashboard receiving and rendering its first telemetry data packet from the ESP32 node. All metric cards display live values, the Chart.js graph begins plotting data points, and the system event log records the data reception event.

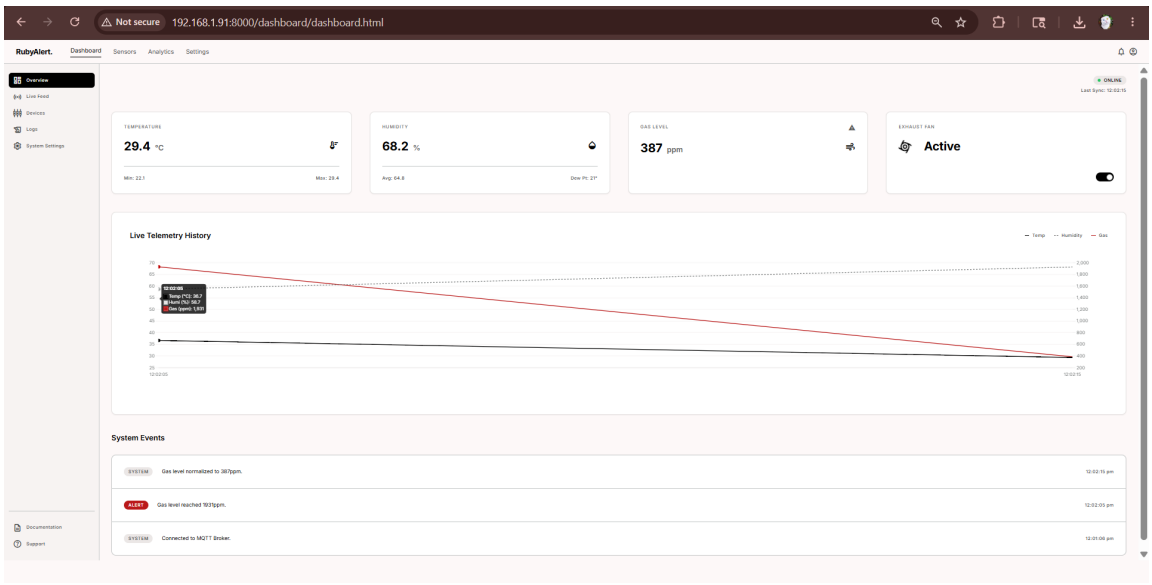

Figure 5.2: Web Dashboard State: Normal Safe Telemetry Stream with Live Chart

5.1.3 Critical Alert State

Figure ?? captures the dashboard during a gas level threshold breach. The Gas Level card transitions from the default white background to a red error state, the warning icon becomes active, and the "CRITICAL THRESHOLD" footer appears. This visual transformation provides immediate situational awareness to the operator.

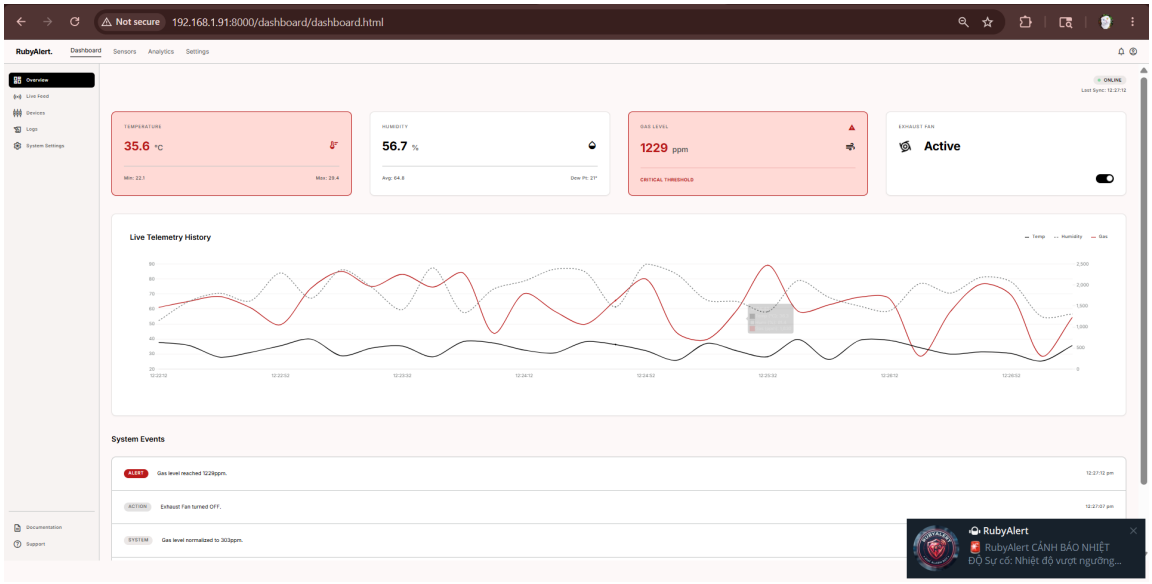

Figure 5.3: Web Dashboard State: Critical Gas Level Threshold Breach Alert

5.2 Telegram Conversational Log Evidence

The actual conversation logs captured on a mobile device illustrate the full spectrum of chatbot capabilities: first data notification, AI-driven device control, and proactive early warning alerts.

5.2.1 First Telemetry Notification

Figure ?? shows the initial system boot message sent to the operator via Telegram, including the dashboard access URLs and the first telemetry data reception confirmation.

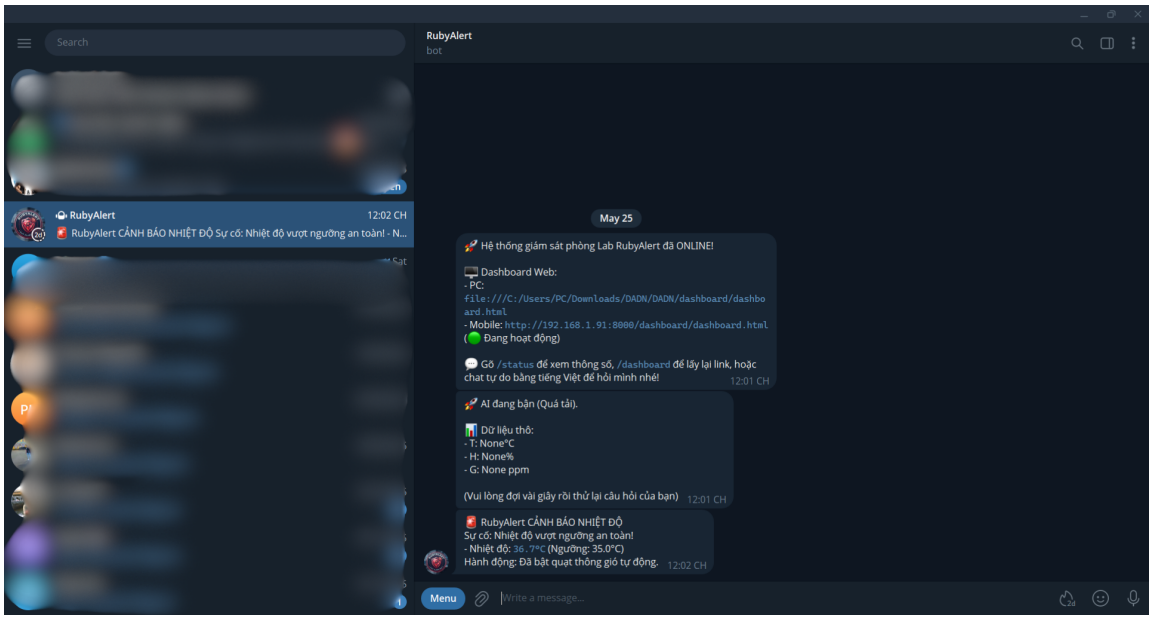

Figure 5.4: Telegram Chatbot: System Boot Notification and First Telemetry Stream

5.2.2 AI Function Calling for Device Control

Figure ?? demonstrates the AI Function Calling workflow. The user sends a natural language instruction (e.g., "bật quạt lên giúp tôi" – "turn on the fan for me"), and the AI model interprets the intent, invokes the `turn_on_fan()` function, and confirms the action with a response message. The fan state change is simultaneously reflected on the Web Dashboard.

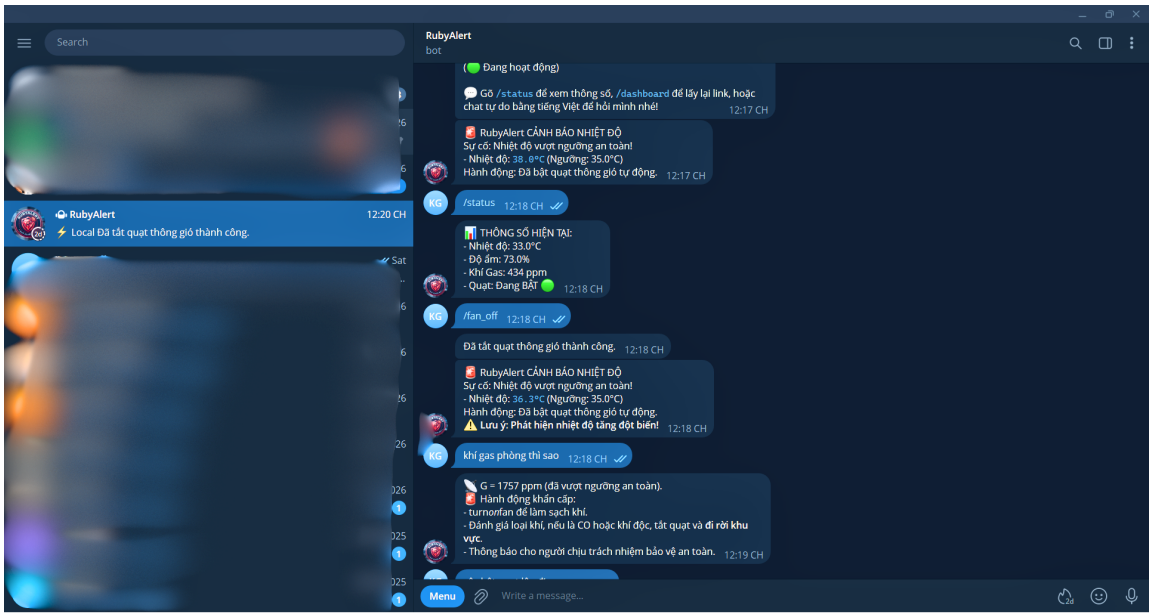

Figure 5.5: Telegram Chatbot: AI-Driven Function Calling for Fan Control

5.2.3 Proactive Early Warning Alerts

Figure ?? captures the system's autonomous alerting behavior. When environmental parameters breach critical thresholds (high temperature and/or gas levels), the system automatically generates and sends structured warning messages with sensor readings, threshold values, and the corrective action taken (fan activation).

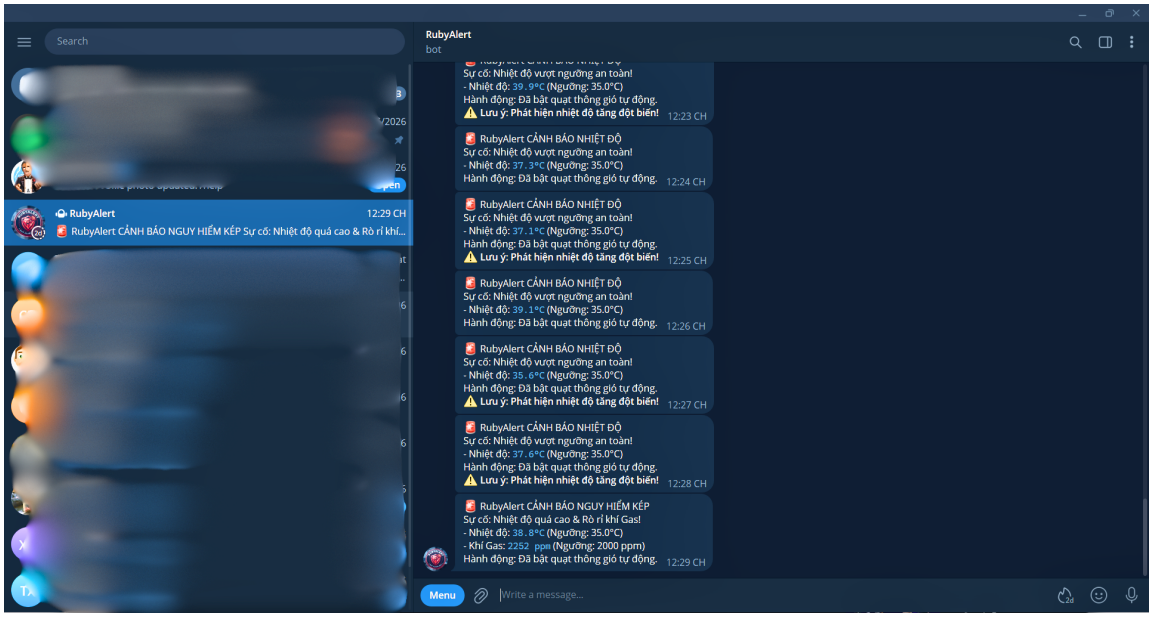

Figure 5.6: Telegram Chatbot: Autonomous Proactive Warning Alerts

5.3 System Verification and Test Cases

The integrated hardware and edge server were subjected to 12 rigorous evaluation scenarios covering all critical system functions. Each test case was executed manually with observed results compared against expected behavior. The complete testing results are presented in Table ??.

ID	Scenario	Action	Expected Response	Actual Response	Status
TC01	System Boot	Start edge server	MQTT broker connection established	Connected to port 1884	Pass
TC02	Overheat	Telemetry $T = 38°C$	Fan ON; Telegram SOS alert	Actuator active; alert sent	Pass
TC03	Hysteresis	Telemetry $T = 33°C$	Fan remains ON (above safe threshold)	Fan active (No chattering)	Pass
TC04	Cooling	Telemetry $T = 30°C$	Fan turns OFF (below 32°C safe)	Fan deactivated at 32°C	Pass
TC05	Gas Leak	Telemetry $G = 2200$ppm	Buzzer active; Fan ON; SOS	Actuators fired; SOS sent	Pass
TC06	Early Leak	Telemetry $G = 1300$ppm	AI Tier 3 proactive fan activation	AI resolved; Fan ON	Pass
TC07	Status Cmd	Send `/status`	Return formatted sensor readings	Real-time stats returned	Pass
TC08	NLP Cmd	Send "turn on fan"	AI function call activates fan	Tool executed successfully	Pass
TC09	Off-topic	Send irrelevant query	AI rejects with refusal prompt	Refusal triggered	Pass
TC10	AI Report	Send `/report`	AI generates safety assessment	Multi-paragraph report	Pass
TC11	Manual Override	Web Dashboard toggle	Auto-rules paused; manual control	Manual control verified	Pass
TC12	Rate Limit	Rapid API requests	Graceful fallback response	Fallback text returned	Pass

Table 5.1: RubyAlert System Verification Matrix (12 Test Cases)

Test Results Summary: All 12 test cases passed successfully, validating that the system meets all functional requirements. The hysteresis logic (TC03–TC04) was particularly important to verify, as it prevents relay chattering that could damage physical components in production deployments. The AI proactive protection (TC06) demonstrated the Tier 3 autonomous supervisor's ability to intervene before traditional hard thresholds are reached.

Chapter 6

Conclusion

6.1 Summary of Achievements

The **RubyAlert** project successfully demonstrated that low-cost microcontrollers, when combined with modern AI services and well-designed software architecture, can be elevated into intelligent, cognitive safety platforms suitable for real-world laboratory environments.

The system was developed as a complete, end-to-end AIoT solution encompassing:

- A physical ESP32 sensory node with MicroPython firmware capable of reading three environmental parameters at 10-second intervals and maintaining persistent MQTT connectivity with automatic reconnection.
- A Python edge server implementing hysteresis automation, rate-of-change spike detection, sensor fault isolation, and manual override priority for reliable, cloud-independent safety guarantees.
- A multi-model AI integration layer supporting four distinct LLM backends (Gemini, Groq, Ollama, Pollinations) with a unified router interface and a Semantic Text Scanner for cross-model device control compatibility.
- A responsive Web Dashboard with live charting, real-time metric cards, visual alert states, and direct fan toggle control via MQTT-over-WebSocket.
- A Telegram Chatbot with 5 structured commands, hybrid local/AI message processing, and autonomous proactive alerting with intelligent cooldown management.

6.2 Key Contributions

The major technical contributions of this work are:

1. **Intelligent Edge Fallbacks:** Integration of local rule-based safety controls (Hysteresis, Rate of Change, Sensor Fault Detection) that override cloud-dependent AI loops during outages, ensuring continuous protection regardless of internet connectivity.
2. **Multi-Model AI Router:** Development of a provider-agnostic AI integration architecture that transparently routes natural language queries to the most appropriate backend, with automatic fallback to free, unlimited services when paid API quotas are exhausted.
3. **Semantic Text Scanner:** Innovation of a keyword-based action extraction mechanism that enables device control through any LLM, regardless of native Function Calling support.
4. **Cognitive AIoT Architecture:** Implementation of a 3-tier AI system (Reactive Assistant, Agentic Actor, Autonomous Supervisor) leveraging state-of-the-art large language models for context-aware laboratory safety management.
5. **High-Availability Design:** Implementation of hybrid keyword/AI processing, token conservation strategies, and AI cooldown timers to enable 24/7 operation at minimal or zero operational cost.

6.3 Future Roadmap

Future extensions of the RubyAlert platform will focus on:

- **Private Broker Security:** Transitioning to TLS/SSL-encrypted MQTT communication with username/password authentication on the Mosquitto Broker to protect data in transit.
- **Persistent Logging:** Integrating an InfluxDB or SQLite database to store telemetry histories for long-term trend analysis, regression modeling, and compliance reporting.
- **On-board Diagnostics Display:** Wiring a physical SSD1306 OLED screen (128×64 pixels, I2C) to the ESP32 to display sensor statistics and connection status during network disconnections.
- **Multi-Room Scalability:** Extending the topic hierarchy (`rubyalert/lab_{n}/telemetry`) to support monitoring multiple laboratory rooms from a single edge server instance.
- **Camera Integration:** Adding an ESP32-CAM module for visual anomaly detection using computer vision models to complement the existing sensor-based monitoring.

- **User Authentication:** Implementing login-based access control on the Web Dashboard to restrict monitoring and control capabilities to authorized personnel.

Bibliography

[1] Google DeepMind, "Gemini API Documentation," 2024. Available online: https://ai.google.dev/docs.

[2] OASIS Standard, "MQTT Version 5.0 – Message Queuing Telemetry Transport," 2019. Available online: https://mqtt.org/.

[3] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.3, 2024. Available online: https://www.espressif.com/en/products/socs/esp32/resources.

[4] Aosong Electronics, "AM2302/DHT22 Digital Temperature and Humidity Sensor Datasheet," 2023.

[5] Zhengzhou Winsen Electronics, "MQ-2 Semiconductor Sensor for Combustible Gas Technical Data," 2022.

[6] Eclipse Foundation, "Eclipse Mosquitto – An Open Source MQTT Broker," 2024. Available online: https://mosquitto.org/.

[7] MicroPython Contributors, "MicroPython – Python for Microcontrollers," 2024. Available online: https://micropython.org/.

[8] Telegram Messenger, "Telegram Bot API Documentation," 2024. Available online: https://core.telegram.org/bots/api.

[9] Pollinations AI, "Free AI Text Generation API," 2024. Available online: https://pollinations.ai/.

[10] Le Hoang Chi Vi, "RubyAlert: AIoT Laboratory Monitoring and Warning System – Source Code Repository," GitHub, 2026. Available online: https://github.com/ukulevi/rubyalert-aiot-lab.